CoThink: Token-Efficient Reasoning via Instruct Models Guiding Reasoning Models

Siqi Fan[1] **Peng Han**[1] **Shuo Shang**[1] **Yequan Wang**[2] **Aixin Sun**[3]
[1]University of Electronic Science and Technology of China
[2]Beijing Academy of Artificial Intelligence, China
[3]Nanyang Technological University, Singapore

Abstract

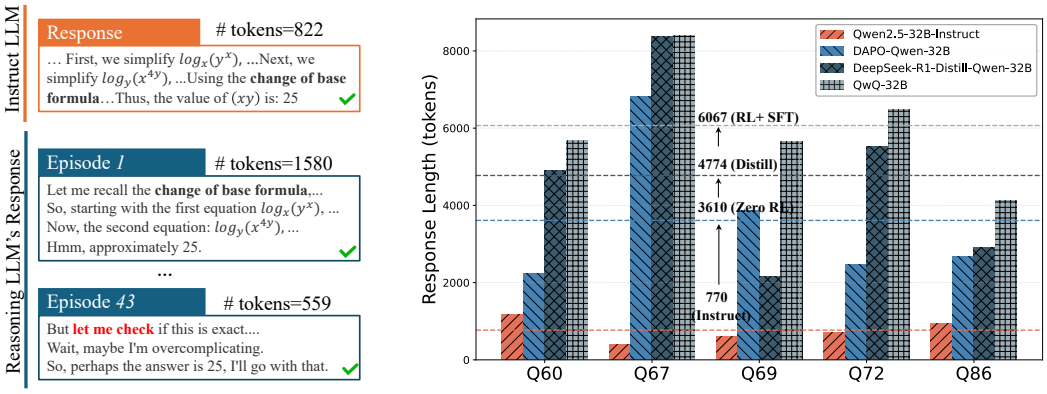
Large language models (LLMs) benefit from increased test-time compute, a phenomenon known as test-time scaling. However, reasoning-optimized models often *overthink* even simple problems, producing excessively verbose outputs and leading to low token efficiency. By comparing these models with equally sized instruct models, we identify two key causes of this verbosity: (1) reinforcement learning reduces the information density of forward reasoning, and (2) backward chain-of-thought training encourages redundant and often unnecessary verification steps. Since LLMs cannot assess the difficulty of a given problem, they tend to apply the same cautious reasoning strategy across all tasks, resulting in inefficient overthinking. To address this, we propose CoThink, an embarrassingly simple pipeline: an instruct model first drafts a high-level solution outline; a reasoning model then works out the solution. We observe that CoThink enables dynamic adjustment of reasoning depth based on input difficulty. Evaluated with three reasoning models DAPO, DeepSeek-R1, and QwQ on three datasets GSM8K, MATH500, and AIME24, CoThink reduces total token generation by 22.3% while maintaining pass@1 accuracy within a 0.42% margin on average. With reference to the instruct model, we formally define *reasoning efficiency* and observe a potential reasoning efficiency scaling law in LLMs.

1 Introduction

Large language models (LLMs) benefit from more test-time compute, an effect known as test-time scaling [1, 2]. Recent language reasoning models, trained via reinforcement learning (RL) with output or process rewards [3, 4], outperform similarly sized instruct models on math problem solving, through multiple rounds of "thinking" and verification. Specifically, in our discussion, a reasoning model refers to a model optimized for complex reasoning inputs (*e.g.,* OpenAI O1 [3], DeepSeek R1 [4]), often exhibiting an internal "thinking process" that involves deeper analysis but with greater computational cost. Reported in many studies [5], these models perform better on more challenging mathematics benchmarks. In comparison, we consider the general instruction-tuned LLM (*i.e.,* Qwen2.5 Instruct [6]) trained via SFT on a broad set of instruction data non-reasoning models, or instruct models. These models are typically effective at handling a wide range of tasks.

Reasoning models often produce much longer and, in many cases unnecessary outputs, even for simple and straightforward questions. For example, on the AIME2024 (an university-level mathematics benchmark), reasoning models produce responses 5 to 10 times longer than the similar sized instruct models, even for questions when both types of models can correctly solve. Figure 1a illustrates an

Preprint. Under review.



(a) Example output for question Q67

(b) #tokens for 5 questions; dotted lines indicate average

Figure 1: Illustration of token lengths for example questions from AIME 2024, where all models successfully answer all these questions: (a) shows answers by Qwen2.5-Instruct-32B (Instruct LLM) and DeepSeek-R1-distill-Qwen-32B (Reasoning LLM) on Q67, (b) plots the total number of tokens in their solutions for 5 questions. Note: Question ID follows the Qwen2.5-Math evaluation format [6], ranging from Q60 to Q89.

example instruct model response alongside the corresponding segmented episodes from a reasoning model. Following [7] we name each round of checking an *Episode*.[1]

For complex problems, lengthy outputs are understandable, as they reflect more "think time", aligning with the model's expected behavior. In fact, human tends to think longer on harder problems to make better decisions [8, 9]. However, verbosity of reasoning models becomes inefficient on simpler problems. This phenomenon has been referred to in recent studies as *overthinking phenomenon* [10, 11]. To address overthinking in reasoning models, prior work has proposed direct strategies imposing token budgets in the prompt [12, 13] or penalizing lengthy responses [14, 15]. However, these methods only superficially limit output length and often fail on complex inputs, where short responses may reflect insufficient "think time".

Through a case study comparing the instruct and reasoning variants within the same model family, we observe that the single and only episode produced by instruct LLMs often demonstrate high-quality results than a typical episode *e.g.,* one thinking/checking process. The instruct LLM achieves comparable accuracy with significantly fewer tokens, a property we refer to as higher *information density*, with respect to number of tokens. The high information density may also suggests that the instruct model is able to give a more serious "thinking" of the problem, although it lacks the capability of verification afterward. A key question is how to leverage the capabilities of the instruct model to mitigate the problem of overthinking.

In problem-solving, people often start by drafting an outline before refining the details. Hence, we can leverage the instruct LLM, for its concise and focused reasoning, to generate a high-level solution outline. Subsequently, a reasoning LLM is employed to elaborate on the details guided by this outline. Drawing inspiration from sketch prompting [16–18], we propose an extremely simple two-stage framework **CoThink**: instruct model to draft a solution outline; then, a reasoning model solves the problem by following the outline. Our intuition is that for simple problems, the instruct model often produces a sufficient outline, requiring only light supplementation from the reasoning model, thereby significantly reducing output length. For more complex problems, the reasoning model naturally activates its full backward checking process to ensure accuracy.

Using Qwen2.5-Instruct-32B as the reference instruct model, we evaluated three reasoning models of the same size *i.e.,* DAPO, DeepSeek-R1-Distill, and QwQ, each trained with different algorithms for enhanced reasoning capabilities. Across three benchmark datasets (GSM8K, MATH500, and AIME24), our experiments show that CoThink reduces total token generation by 22.3% while maintaining pass@1 accuracy within 0.42% margin on average across 9 the test cases. CoThink also

[1]Note that there is no standard criterion for clearly segmenting a response into episodes. In this work, we use regular expressions with keywords such as "let me verify", "let me check", and "on second thought".

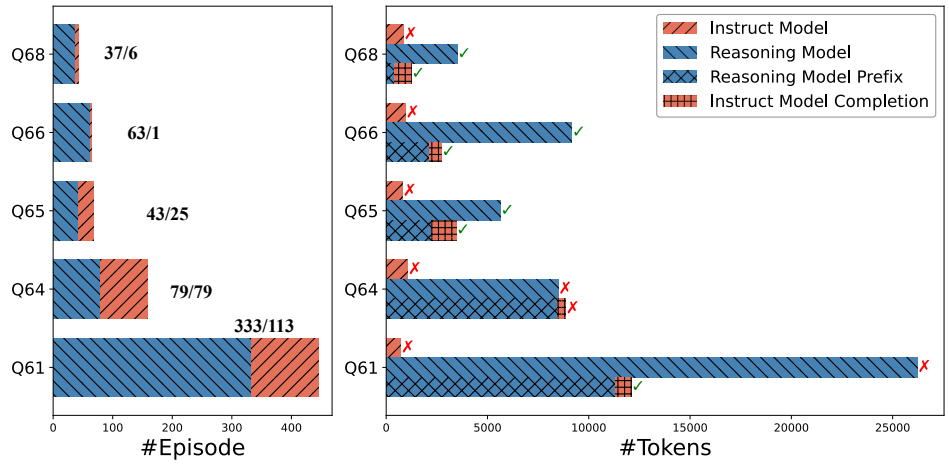


Figure 2: We present five AIME24 questions that the instruct model (Qwen2.5-32B-Instruct) fails to answer on its own. For each question, we prepend reasoning episodes generated by the DeepSeek-R1-Distill-Qwen-32B model as context, and test whether this helps the instruct model arrive at the correct answer. **Left**: For each question, we report the total number of reasoning episodes generated, as well as the minimum number required for the instruct model to produce the final answer. **Right**: For the same questions, we show token counts for (i) the original (failed) output from the instruct model, (ii) the reasoning model's output, (iii) the total tokens from the reasoning episodes used as context, and (iv) whether the final answer was correct.

demonstrated competitiveness against baseline solutions such as Best-of-N and NoThinking, which are designed to improve token efficiency in reasoning models.

For evaluation purposes, we formally define *reasoning efficiency* as the ratio of token efficiency between a reasoning model and its instruct counterpart. Experimental results suggest the existence of a potential reasoning efficiency scaling law in LLMs. However, all the evaluated reasoning models demonstrate significantly lower reasoning efficiency compared to the instruct model.

A key insight attributed to CoThink is that reasoning models tend to be verbose due to low forward information density, induced by reinforcement learning, and repetitive checking behavior from backward Chain-of-Thought training. These patterns result in unnecessary verbosity and reduced token efficiency. Because CoThink is a simple form of prompt engineering and does not alter model behavior or interfere with the reasoning process, it can be easily integrated with other solutions to further enhance reasoning efficiency.

2 Case Study on AIME24

As a case study, we evaluate four representative 32B models on the AIME24 dataset: one general-purpose instruct model (Qwen2.5-Instruct) and three reasoning models trained with distinct algorithms (Table 1). Together, these models cover various distinct supervision strategies, enabling us to isolate the effects of different training data and algorithms on performance. Question formatted using the official prompt template with HuggingFace's recommended `add_chat_template` function [2], and insert an additional `<Think>` tag to encourage step-by-step reasoning. To ensure fairness and reproducibility, we use HuggingFace's official `Math-Verify` [3] to validate all generated answers.

We first compare the instruct model and one representative reasoning model (DeepSeek-R1-Distill) on the AIME24 dataset. Overall, DeepSeek-R1-Distill achieves significantly higher accuracy than Qwen2.5-Instruct. Their outputs fall into three categories: (i) On 5 out of 30 questions, both models provide correct answers. The instruct model takes a concise and direct approach, while the reasoning model follows similar logic with more verbose steps and occasional backward checking. (ii) On 16 questions, only the reasoning model produces the correct answer, typically by identifying and

[2] `https://huggingface.co/docs/transformers/main/en/chat_templating`

[3] https://github.com/huggingface/Math-Verify

Table 1: Model configurations and training methodologies. All models have 32B parameters. "Bwd CoT" means training with backward chain-of-thought data.

Model	SFT	RL	Bwd CoT	Training Focus
Qwen2.5-Instruct [6]	✓	✗	✗	Standard Instruction Tuning
DAPO [19]	✗	✓	✓	Reinforcement Learning (RL) for Reasoning
DeepSeek-R1-Distill [4]	✓	✗	✓	Distillation-based reasoning
QwQ [20]	✓	✓	✓	Combined SFT and RL

correcting errors through backward checking, a capability the instruct model lacks. (iii) On 9 questions, both models fail. The instruct model gives brief incorrect answers, while the reasoning model attempts more elaborate but still unsuccessful reasoning, often due to missing knowledge. Notably, there are no questions where the instruct model is able to solve but the reasoning model fails. Nevertheless, the reasoning model consistently generates significantly more tokens for all answers, suggesting a higher computational cost.

Observation 1 (Instruct Model Shows High Token Efficiency) *Compared to reasoning models, the instruct model produces significantly shorter responses, especially for questions it can answer correctly.*

Now, based on the three categories, we take a closer look at how reasoning models and instruct model differ in their output patterns and abilities.

Figure 1b highlights the five AIME24 questions (Q60, Q67, Q69, Q72, Q86) correctly solved by all four models listed in Table 1. The average output lengths on these questions increase progressively: 770 (Instruct) → 3610 (DAPO) → 4774 (DeepSeek-R1-Distill) → 6067 (QwQ). Given that all models succeed, we consider the instruct model a baseline, representing solutions that require minimal reasoning effort and a substantially smaller token count.

The instruct model, which lacks backward CoT training, typically generates only a single forward reasoning episode without backward checking. In contrast, reasoning models often produce multiple episodes. As illustrated in Figure 1a, even when both models apply similar problem-solving knowledge and logical steps, *e.g.,* simplifying the first and second equations and using the change of base formula, the reasoning models are markedly more verbose. For example, the instruct model uses 822 tokens to arrive at the answer, while the reasoning model uses about 1580 tokens in its first episode and then continues with additional episodes for repeated backward checking.

We attribute this verbosity to reinforcement learning, which reduces per-step information density and encourages conservative, lengthier responses. Our finding is consistent with [21]: while RL can improve *pass@1*, it promotes verbose generation. This tendency is preserved and amplified during distillation, leading models like DeepSeek-R1-Distill and QwQ to produce longer outputs than even the pure-RL model DAPO.

Observation 2 (Not All Reasoning Episodes Are Necessary) *Given reflective episodes from reasoning model as background context, instruct models can solve previously incorrect problems with comparable accuracy, using far fewer episodes and tokens.*

Reasoning models often generate multiple episodes, with episodes 2 through N primarily serving as backward checks to re-verify earlier steps. This behavior stems from training on backward Chain-of-Thought (CoT) data. To evaluate whether these additional episodes are essential for problem-solving, we conduct a case study using the first five AIME24 questions that the instruct model (Qwen2.5-Instruct) fails to solve (Q61, Q64, Q65, Q66, Q68). Notably, Q65 is also answered incorrectly by the reasoning model (DeepSeek-R1-Distill). For each of these five questions, we prepend the reasoning model's episodes as context and re-evaluate the instruct model's performance.

As shown in Figure 2, the instruct model only needs a small subset of the reasoning episodes—on average 27.5%—to arrive at a correct answer. More strikingly, its generated correct answer requires just 11.9% of the output tokens used by the reasoning model. This indicates that the instruct model possesses the necessary problem-solving knowledge but lacks backward checking mechanism. Once provided with few reflective episodes, it is capable of solving problems more efficiently than reasoning

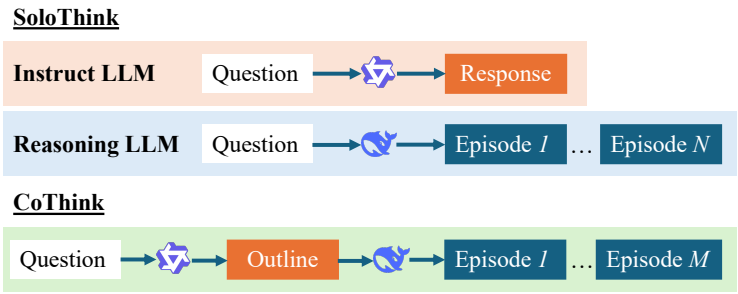


Figure 3: An illustration of the CoThink two-stage framework compared with its SoloThink counterparts using either an instruct model or a reasoning model.

models. Conversely, it is infeasible to predict how many contextual reasoning episodes an instruct model requires to produce a correct answer, or eventually fails to answer.

3 CoThink

Our case study shows that instruct models solve problems more efficiently but struggle with complex reasoning due to a lack of backward checking. At first glance, a natural solution is to delegate easy tasks to instruct models and reserve harder ones for reasoning models. Recent efforts such as hybrid reasoning [22–27] aim to solve this adaptively. For example, Qwen3 [22] and NoThinking [23] implement hard-coded strategies that switch model behavior based on perceived input difficulty.

The fundamental difficulty lies in identifying problem difficulty before solving. Users, and models alike, typically cannot tell how hard a question is until they begin working on it. During the prefill phase, LLMs treat all inputs equally, lacking the means to adapt their reasoning strategy. In practice, difficulty is not a static property of the input—it emerges dynamically during generation. Some problems are solved in a few steps; others require extended reasoning and self-correction. This makes preemptive difficulty assessment inherently unreliable. Prior work often resorts to handcrafted difficulty labels or controlled settings.

Interestingly, by providing the episodes produced by reasoning models as context to instruct model, our case study already demonstrate the cooperation between the two kinds of models. In this context, we propose CoThink, a dynamic, two-stage solution process, as illustrated in Figure 3.

Stage 1: Outline Generation by Instruct Model The instruct model generates a concise outline without solving the problem, leveraging its high token efficiency in straightforward reasoning to assist the reasoning model. The prompts are detailed below:

System Prompt:

You are a reasoning strategist.
Your job is to break down a complex problem into 2–4 high-level reasoning steps.
Focus only on outlining the general approach or strategy.
Do not include any numbers, formulas, or final answers.
Avoid specific calculations or details—only describe the logic behind solving the problem.

User Prompt:

Please break down the following problem.

Problem: {problem}

Stage 2: Backward Verification by Reasoning Model By following the high-density outline from the instruct model, the reasoning model efficiently verifies and completes it using fewer tokens.

User Prompt:

Use only the following steps to solve the problem. Do not change or add steps. Show the work for each step briefly, and place the final answer in \\boxed{}.
Problem: {problem}
Steps: {*outline generated by instruct model*}

Table 2: Evaluation Benchmarks.

Dataset	Level of difficulty	#Samples	#Tokens in ground truth solutions
GSM8K	Primary	1,319	48–1,070
MATH500	High school	500	45–3,360
AIME24	University	30	284–4,010

Observe that the proposed CoThink represents a "reverse" setting compared to our earlier case study in Figure 2. Previously, we fed the reasoning episodes produced by the reasoning model into the instruct model. In CoThink, we instead feed the output of the instruct model into the reasoning model. This approach avoids the challenge of predicting the correct number of reasoning episodes and reduces the overhead of lengthy reasoning. Interestingly, *CoThink is a more intuitive setup*. For simple tasks, the instruct model's outline is often correct or nearly correct, allowing the reasoning model to converge quickly with minimal effort. For harder tasks, the outline provides a structured starting point, enabling the reasoning model to apply backward checking and ensure correctness—avoiding unstructured trial-and-error from scratch.

4 Experiments

We evaluate CoThink using the same set of LLMs as in our case study, see Table 1. They include one instruct model (Qwen2.5-Instruct-32B) and three reasoning models trained with different algorithms *i.e.,* DAPO, DeepSeek-R1-Distill, QwQ. These models cover diverse supervision strategies, allowing a comprehensive evaluation of CoThink's performance.

4.1 Experimental Setup

We evaluate on three math benchmarks, GSM8K, MATH500, and AIME24, covering increasing difficulty and chronological release [5] (see Table 2). GSM8K consists of relatively simple grade-school level problems with short solutions, while MATH500 includes more complex high school competition problems. AIME24 is the most challenging, featuring problems from prestigious high school mathematics competitions with significantly longer solutions. This setup helps us validate the insight from the case study, that is whether the instruct model can effectively guide the reasoning model to perform token-efficient inference when handling different kinds of questions. All prompts used across models adhere to the standard Hugging Face template recommendations as Section 2 outlined.

Baseline. We compare CoThink against three methods where models solve problems independently, without external outline guidance: (1) *SoloThink:* A single model solves the problem step by step; this reflects typical usage in practical settings. We evaluate both instruct and reasoning models in this context. (2) *NoThinking* [23]: The reasoning model assistant side is prompted with "Okay, I think I have finished thinking." to skip the thinking process and generate the answer directly. (3) *Best of N:* The reasoning model is prompted to generate multiple solutions ($N = 5$), and the shortest one is selected as the final solution.

Evaluation Metrics. For evaluation, we consider both model accuracy and computational efficiency. For accuracy, we use ***Pass@1***—the percentage of problems for which the first sampled solution is correct, as verified by Hugging Face's official tool, `Math-Verify`[4].

To assess efficiency, we first quantify computational cost which can be measured in various ways *e.g.,* inference time and the number of reasoning steps, depending on the application setting [28]. In our setting, since all models generate solutions via sampling with a fixed temperature of 0.6, we use token-level statistics to estimate computational cost. Specifically, we report *Generation Token Length*

[4] `https://github.com/huggingface/Math-Verify`

Table 3: Accuracy and efficiency of different reasoning methods across three datasets. The instruct model serves as the baseline reference for reasoning efficiency η. For CoThink in each setting, improvements over SoloThink of the reasoning model are marked in green, declines in red.

Method	GSM8K				MATH500				AIME24			
	Pass@1↑	#Tokens↓	τ ↑	η ↑	Pass@1↑	#Tokens↓	τ ↑	η ↑	Pass@1↑	#Tokens↓	τ ↑	η ↑
Instruct model: Qwen2.5-Instruct-32B (as a reference)												
SoloThink	96	309	31.07	100	82	505	16.24	100	16.7	1,077	1.56	100
Reasoning model: DAPO-Qwen-32B (zero RL on Qwen2.5-32B)												
SoloThink	98	**510**	**19.22**	**61.99**	67	2,025	3.31	20.38	46.7	6,639	0.70	45.36
Best-of-N	98	2,611	3.75	12.11	65	11,464	0.57	3.49	**60**	30,210	0.20	12.81
NoThinking	98	516	18.99	61.27	68	2,742	2.48	15.27	46.7	6,965	0.67	43.24
CoThink	+0.0% 98	+6.3% 542	18.08	58.33	+35.8% **91**	-15.7% **1,707**	**5.33**	**32.83**	-14.3% 40	-29.4% **4,686**	**0.90**	**58.34**
Reasoning model: DeepSeek-R1-Distill-Qwen-32B (Distilled from Qwen2.5-32B)												
SoloThink	94.5	823	11.48	37.04	94	3,199	2.94	18.10	70	10,208	**0.69**	**44.22**
Best-of-N	**95.5**	4,295	2.22	7.17	**97**	15,857	0.61	3.77	**76.7**	57,943	0.13	8.54
NoThinking	**95.5**	**449**	**21.27**	**68.61**	89	2,809	3.17	19.51	63.3	11,070	0.57	36.88
CoThink	-2.1% 92.5	-35.7% 529	17.49	56.41	-2.1% 92	-36.6% **2,027**	**4.54**	**27.94**	-19.0% 56.7	-12.5% **8,937**	0.63	40.92
Reasoning model: QwQ (SFT + RL on Qwen2.5-32B)												
SoloThink	**97.5**	1,602	6.09	19.63	**98**	3,933	2.49	15.35	80	13,977	0.57	36.91
Best-of-N	**97.5**	8127	1.20	3.87	97	18,887	0.51	3.16	80	68,605	0.12	7.52
NoThinking	95	1,679	5.66	18.25	96	4,047	2.37	14.61	80	14,590	0.55	35.36
CoThink	-3.1% 94.5	-41.8% **933**	**10.13**	**32.67**	-3.1% 95	-19.1% **3,183**	**2.98**	**18.38**	+4.1% **83.3**	-16.2% **11,717**	**0.71**	**45.85**

(**#Tokens**): the average number of tokens generated by the model to solve each problem across the entire process.[5]

Based on both accuracy and #Tokens reflecting computational cost, we define the ***Token Efficiency*** of a model M on an evaluation dataset D, denoted by $\tau(M, D)$, as the ratio of solution quality $Q(D)$ (*i.e.,* model accuracy) to the number of tokens used as the computational cost $C(D)$. A model with high token efficiency produces high-quality outputs using fewer tokens.

$$\tau(M, D) = \frac{Q_M(D)}{C_M(D)} \tag{1}$$

Existing methods [29] for evaluating the efficiency of reasoning models often focus on the trade-off between computational cost and performance—an idea captured by our token efficiency metric. In this work, we further qualify reasoning efficiency. We assume that the computation used by an instruct model to solve a problem reflects the baseline effort required for that input. If a reasoning model solves the same problem but requires more computation, the extra cost can be interpreted as potential redundancy.

Specifically, if the model family includes two variants, an instruct model and a reasoning model, of the same parameter size, denoted M_I and M_R respectively—we define the ***Reasoning Efficiency*** $\eta(M_R, M_I)$ as the ratio of the token efficiency of the reasoning model to that of the instruct model:

$$\eta(M_R, M_I) = \frac{\tau(M_R, D)}{\tau(M_I, D)} = \frac{Q_R(D) \cdot C_I(D)}{Q_I(D) \cdot C_R(D)} \tag{2}$$

Intuitively, reasoning becomes more efficient either by improving solution quality at the same cost, or by reducing cost while maintaining quality.

4.2 CoThink against Baselines

We evaluate against three baselines—SoloThink, Best-of-N, and NoThinking—across three reasoning models (DAPO, DeepSeek-R1-Distill, QwQ) and three math reasoning benchmarks (GSM8K, MATH500, AIME24), resulting in 9 experimental settings. Reported in Table 3, in terms of accuracy, CoThink achieves the best in 3 settings, while Best-of-N leads in 4, and SoloThink in 2. For total tokens generated, CoThink ranks first in 7 out of 9 settings, followed by SoloThink and NoThinking with one win each. Regarding token efficiency (τ) and reasoning efficiency (η), CoThink outperforms all baselines in 6 settings; SoloThink wins 2, and NoThinking wins 1. Notably, compared to each

[5]Note: Best-of-N involves multiple sampled candidates, and CoThink includes outline tokens, both of which contribute to the reported Generation Token Length. For reference, the instruct model produces an average of 78, 154, and 264 outline tokens on GSM8K, MATH500, and AIME24, respectively.

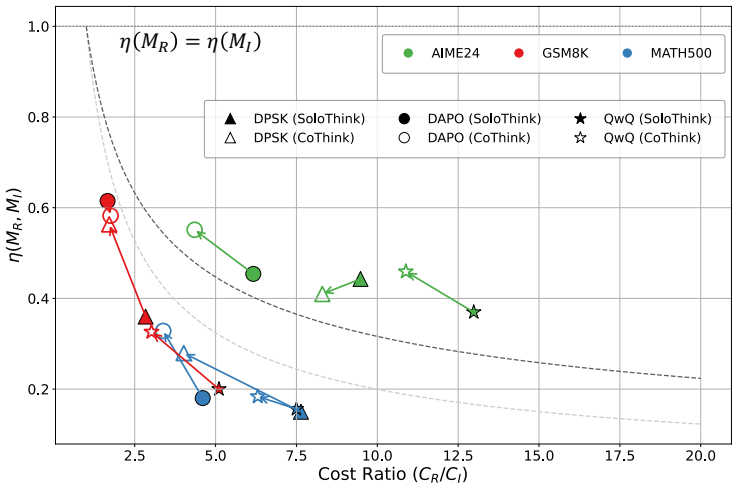


Figure 4: Reasoning efficiency comparison between SoloThink and CoThink. Each model is represented by a specific marker shape, and each dataset by a distinct color. The dashed gray lines correspond to hypothesized efficiency scaling law with assumed scaling exponents $\beta = 0.3$ and $\beta = 0.5$ for reference.

model's own SoloThink, on average, CoThink reduces total token usage by 22.3%, reaching up to 41.8% in some cases, while maintaining accuracy within a 0.42% margin. This efficiency gain is achieved without requiring prior estimation of problem difficulty, making CoThink a practical choice for many scenarios.

CoThink improves efficiency across all three reasoning models, with the overall trend being: QwQ > DeepSeek-R1-Distill > DAPO. QwQ and DeepSeek benefit from SFT training, making them better at following the outline instructions in CoThink. In contrast, DAPO, trained via RL from a base model, follows less consistently. However, in tasks like MATH500—where DAPO performs poorly but the instruct model does well—CoThink shows strong guiding ability. These consistent efficiency gains across diverse reasoning models highlight the generality and robustness of CoThink in improving reasoning efficiency. In particular, on the most challenging dataset, AIME24, CoThink benefits from the strongest reasoning model, QwQ, to achieve the highest pass@1 accuracy along with the best token and reasoning efficiency, demonstrating its potential to complement models with strong reasoning capabilities.

4.3 Hypothesized Reasoning Efficiency Scaling Law

Language model performance typically follows a test-time scaling law [1, 30], where response quality improves sublinearly with increased cost: $Q(C) \propto C^{\beta}$, with $\beta < 1$. This reflects a phenomenon of diminishing returns—achieving linear gains in quality requires exponential increases in cost. Under this assumption, our reasoning efficiency metric can be approximated as:

$$\eta \approx \left(\frac{C_R}{C_I}\right)^{\beta} \tag{3}$$

Figure 4 plots the changes in the reasoning efficiency η of the three reasoning models across three datasets, from each model's SoloThink to that of after applying CoThink. For illustration purpose, we also plot two curves with $\beta = 0.3$ and 0.5 respectively.

When the metric $\eta = 1$, it indicates that the token efficiency of the reasoning model matches that of the instruct model. As shown, all reasoning models have efficiencies below 1. According to the trend from Equation 3, linear improvements in performance require exponential increases in computational cost. For complex tasks, the reasoning model is expected to exhibit relatively higher reasoning efficiency compared to simpler tasks. This is because simple problems often lead to overthinking, consuming more tokens than necessary relative to the instruct model. In contrast, complex tasks inherently require more computation and repeated checking, where the reasoning model's strengths

are better utilized—especially in cases where the instruct model struggles. Furthermore, the hollow markers representing CoThink mostly show improvements in reasoning efficiency, consistent with the directions indicated by the arrows.

5 Related Work

Token efficiency refers to the problem-solving quality achieved per unit of computation [29], capturing the trade-off between performance and cost. Different models show opposite issues: instruct models often underthink on hard tasks, while reasoning models tend to overthink, generating redundant steps even for simple ones [31, 11, 10, 32]. To mitigate overthinking, some works limit output length via prompts [12, 13, 33, 34], encourage early stopping [35–37], RL with length penalty [15, 38, 39] or SFT on short-solution [14, 40, 41]. However, these approaches tend to reduce token usage only superficially, and on more challenging tasks, shorter outputs might compromise accuracy by limiting the necessary "thinking time". A more natural way is to assign easy tasks to instruct models and harder ones to reasoning models. Recent hybrid reasoning methods [22–27] adaptively assign tasks based on perceived difficulty—for example, Qwen3 [22] and NoThinking [23] use hard-coded switching rules.

A key challenge lies in whether LLMs can perceive problem difficulty. As LLMs are often treated as black boxes, prior work has explored this indirectly through interpretability methods. Some analyze attention patterns show that CoT helps LLMs reason on harder problems [42–44]. Others disrupt CoT via prompt perturbations [45, 46], revealing a disconnect between what the model "knows" and what it "says": on simple problems, generation often lacks faithfulness, whereas complex tasks trigger more genuine reasoning. Several studies also observe that correct answers often appear early in generation [35, 47, 48], suggesting the potential to stop generation once confidence is high. However, this may lead to hallucinations, where the model confidently produces incorrect outputs. We hypothesize that LLMs treat all inputs equally during the prefill phase, and difficulty emerges dynamically during generation. For example, models may find the correct answer early but keep generating redundant steps due to learned patterns. This dynamic assessment remains underexplored and lacks accurate evidence.

We propose an extremely simple two-stage collaborative pipeline inspired by sketch prompting engineering [16, 49, 18]. Several concurrent works explore related directions. Thought Manipulation [25] inserts a pre-generated CoT between the reasoning model's <think> tag, allowing the model to better leverage external reasoning. Scot [50] runs a lightweight model in parallel to draft multiple CoT sketches, from which a reasoning model selects.

In contrast, our approach focuses on transferring the dense forward reasoning capabilities of instruction-tuned models to reasoning models by using a high-quality outline as the starting point. Moreover, our method is highly practical and deployment-friendly—it requires no modification to model architecture and simply inserts the outline into the reasoning model's user prompt, enabling low-cost improvements in reasoning quality suitable for real-world applications.

6 Conclusion

Large language models often overthink simple tasks and underthink complex ones due to treating all inputs uniformly during inference. We identify reinforcement learning and backward CoT as key contributors to verbose, low-density reasoning in specialized reasoning models. To address this, we propose CoThink, a simple two-stage pipeline that leverages a instruct model for outlining and a reasoning model for refinement. This approach improves the reasoning model's efficiency while maintaining high accuracy across tasks and models.

In this study, we also formally define reasoning efficiency based on token efficiency. Our results reveal consistent trends across model types and datasets, suggesting the existence of a potential reasoning efficiency scaling law in LLMs. This metric may offer a unified basis for comparing reasoning capabilities across models and datasets. While still speculative, it provides a useful perspective for examining future trade-offs between reasoning accuracy and computational cost.

References

[1] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[2] Nishad Singhi, Hritik Bansal, Arian Hosseini, Aditya Grover, Kai-Wei Chang, Marcus Rohrbach, and Anna Rohrbach. When to solve, when to verify: Compute-optimal problem solving and generative verification for llm reasoning. *arXiv preprint arXiv:2504.01005*, 2025.

[3] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[4] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[5] Yixin Cao, Shibo Hong, Xinze Li, Jiahao Ying, Yubo Ma, Haiyuan Liang, Yantao Liu, Zijun Yao, Xiaozhi Wang, Dan Huang, et al. Toward generalizable evaluation in the llm era: A survey beyond benchmarks. *arXiv preprint arXiv:2504.18838*, 2025.

[6] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[7] Yuxiao Qu, Matthew YR Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. Optimizing test-time compute via meta reinforcement fine-tuning. *arXiv preprint arXiv:2503.07572*, 2025.

[8] Jonathan St BT Evans. Heuristic and analytic processes in reasoning. *British Journal of Psychology*, 75(4):451–468, 1984.

[9] Daniel Kahneman. Maps of bounded rationality: Psychology for behavioral economics. *American economic review*, 93(5):1449–1475, 2003.

[10] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. Do not think that much for 2+ 3=? on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*, 2024.

[11] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.

[12] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

[13] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

[14] Wenkai Yang, Shuming Ma, Yankai Lin, and Furu Wei. Towards thinking-optimal scaling of test-time compute for llm reasoning. *arXiv preprint arXiv:2502.18080*, 2025.

[15] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.

[16] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.

[17] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting llms for efficient parallel generation. *arXiv preprint arXiv:2307.15337*, 2023.

[18] Luca Beurer-Kellner, Mark Niklas Müller, Marc Fischer, and Martin Vechev. Prompt sketching for large language models. *arXiv preprint arXiv:2311.04954*, 2023.

[19] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[20] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025.

[21] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[22] Qwen Team. Qwen3, April 2025.

[23] Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking. *arXiv preprint arXiv:2504.09858*, 2025.

[24] Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, Li Dong, Xingxing Zhang, Tengchao Lv, Lei Cui, and Furu Wei. Think only when you need with large hybrid-reasoning models. *arXiv preprint arXiv:2505.14631*, 2025.

[25] Yule Liu, Jingyi Zheng, Zhen Sun, Zifan Peng, Wenhan Dong, Zeyang Sha, Shiwen Cui, Weiqiang Wang, and Xinlei He. Thought manipulation: External thought can be efficient for large reasoning models. *arXiv preprint arXiv:2504.13626*, 2025.

[26] Haotian Luo, Haiying He, Yibo Wang, Jinluan Yang, Rui Liu, Naiqiang Tan, Xiaochun Cao, Dacheng Tao, and Li Shen. Adar1: From long-cot to hybrid-cot via bi-level adaptive reasoning optimization. *arXiv preprint arXiv:2504.21659*, 2025.

[27] Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. Adaptthink: Reasoning models can learn when to think. *arXiv preprint arXiv:2505.13417*, 2025.

[28] Mostafa Dehghani, Anurag Arnab, Lucas Beyer, Ashish Vaswani, and Yi Tay. The efficiency misnomer. *arXiv preprint arXiv:2110.12894*, 2021.

[29] Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, et al. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond. *arXiv preprint arXiv:2503.21614*, 2025.

[30] OpenAI. Learning to reason with language models, 2024. Accessed: 2025-05-19.

[31] Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. Efficient reasoning models: A survey. *arXiv preprint arXiv:2504.10903*, 2025.

[32] Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Linfeng Song, Dian Yu, Juntao Li, Zhuosheng Zhang, et al. Thoughts are all over the place: On the underthinking of o1-like llms. *arXiv preprint arXiv:2501.18585*, 2025.

[33] Simon A Aytes, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.

[34] Yuchen Yan, Yongliang Shen, Yang Liu, Jin Jiang, Mengdi Zhang, Jian Shao, and Yueting Zhuang. Inftythink: Breaking the length limits of long-context reasoning in large language models. *arXiv preprint arXiv:2503.06692*, 2025.

[35] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they're right: Probing hidden states for self-verification. *arXiv preprint arXiv:2504.05419*, 2025.

[36] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Zheng Lin, Li Cao, and Weiping Wang. Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*, 2025.

[37] Guochao Jiang, Guofeng Quan, Zepeng Ding, Ziqin Luo, Dixuan Wang, and Zheng Hu. Flashthink: An early exit method for efficient reasoning. *arXiv preprint arXiv:2505.13949*, 2025.

[38] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

[39] Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.

[40] Heming Xia, Yongqi Li, Chak Tou Leong, Wenjie Wang, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.

[41] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 24312–24320, 2025.

[42] Tobias Schnabel, Kiran Tomlinson, Adith Swaminathan, and Jennifer Neville. Lost in transmission: When and why llms fail to reason globally. *arXiv preprint arXiv:2505.08140*, 2025.

[43] Benjamin L Edelman, Surbhi Goel, Sham Kakade, and Cyril Zhang. Inductive biases and variable creation in self-attention mechanisms. In *International Conference on Machine Learning*, pages 5793–5831. PMLR, 2022.

[44] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

[45] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.

[46] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, et al. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.

[47] Pranjal Aggarwal, Aman Madaan, Yiming Yang, et al. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with llms. *arXiv preprint arXiv:2305.11860*, 2023.

[48] Amir Taubenfeld, Tom Sheffer, Eran Ofek, Amir Feder, Ariel Goldstein, Zorik Gekhman, and Gal Yona. Confidence improves self-consistency in llms. *arXiv preprint arXiv:2502.06233*, 2025.

[49] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021.

[50] Jikai Wang, Juntao Li, Lijun Wu, and Min Zhang. Efficient reasoning for llms through speculative chain-of-thought. *arXiv preprint arXiv:2504.19095*, 2025.